



Instituto Politécnico Nacional
Unidad Profesional
Interdisciplinaria de Ingeniería y
Ciencias Sociales y
Administrativas



Prácticas

Tercer Parcial

Integrantes	No. Lista
Martínez Santiago Daniel	21
Galindo Cantillo Franco	12
Díaz Gasca Sofia	10
Pedro Zenon Monserrat	32

Profesor: Yventz Entzana Garduño

Secuencia: 2NM23

Repositorio de GitHub:

<https://github.com/Monserrat2007/Programas-segundo-departamental-Estructura-de-datos>

Ejercicio del primer parcial: Calculadora Básica con archivos CSV, TXT, XML y JSON y Lector de archivos

Descripción del programa:

El programa consiste en una calculadora básica desarrollada en C++ capaz de realizar las operaciones aritméticas fundamentales: suma, resta, multiplicación y división. El usuario ingresa dos números y selecciona la operación deseada mediante un menú.

Una vez calculado el resultado, el programa genera automáticamente cuatro archivos diferentes para almacenar la información de la operación realizada:

- CSV
- TXT
- XML
- JSON

Además, se desarrolló un segundo programa independiente denominado Lector, cuya función es permitir al usuario seleccionar uno de los archivos generados y visualizar su contenido en pantalla.

El objetivo principal de la práctica fue aplicar conceptos de programación orientada a objetos y manejo de archivos en distintos formatos.

Descripción de clases:

Programa Generador (Calculadora)

Clase IngresarNum.h

Define los atributos necesarios para almacenar los dos números ingresados por el usuario y declara la función de solicitarlos.

```
1  #ifndef INGRESARNUM_H
2  #define INGRESARNUM_H
3
4  class IngresarNum
5  {
6  public:
7      double n1;
8      double n2;
9
10     void pedirNum();
11 };
12
13 #endif
```

Clase IngresarNum.cpp

Implementa la función pedirNum(), la cual solicita al usuario el primer y segundo número mediante el teclado.

```
1  #include <iostream>
2  #include "IngresarNum.h"
3
4  using namespace std;
5
6  void IngresarNum::pedirNum()
7  {
8      cout << "Ingrese el primer numero: ";
9      cin >> n1;
10     cout << "Ingrese el segundo numero: ";
11     cin >> n2;
12 }
```

Clase Operación.h

Define la variable que almacenará la opción seleccionada por el usuario y declara la función para mostrar el menú.

```
1  #ifndef OPERACION_H
2  #define OPERACION_H
3
4  class Operacion
5  {
6  public:
7      int opcion;
8      void mostrarMenu();
9  };
10
11 #endif
```

Clase Operación.cpp

Implementa el menú de operaciones disponibles y almacena la opción seleccionada.

```
1  #include <iostream>
2  #include "Operacion.h"
3
4  using namespace std;
5
6  void Operacion::mostrarMenu()
7  {
8      cout << "Seleccione una operacion: " << endl;
9      cout << "1. Suma" << endl;
10     cout << "2. Resta" << endl;
11     cout << "3. Multiplicacion" << endl;
12     cout << "4. Division" << endl;
13
14     cin >> opcion;
15 }
```

Clase CalculadoraB.h

Define los atributos necesarios para almacenar el resultado de la operación y el nombre de la operación realizada.

```
1  #ifndef CALCULADORAB_H
2  #define CALCULADORAB_H
3
4  #include "IngresarNum.h"
5  #include <string>
6
7  class CalculadoraB
8  {
9  public:
10     double resultado;
11     std::string operacion;
12
13     void calcular(IngresarNum e, int opcion);
14 };
15
16 #endif
```

Clase CalculadoraB.cpp

Implementa la lógica de la calculadora.

Funciones realizadas:

- Suma
- Resta
- Multiplicación
- División
- Validación de división entre cero
- Generación de archivos CSV, TXT, XML y JSON
- Mensaje de confirmación

```

1  #include "CalculadoraB.h"
2  #include <iostream>
3  #include <fstream>
4
5  using namespace std;
6
7  void CalculadoraB::calcular(IngresarNum e, int opcion)
8  {
9      switch (opcion)
10     {
11         case 1:
12             resultado = e.n1 + e.n2;
13             operacion = "Suma";
14             cout << "Suma: " << resultado << endl;
15             break;
16
17         case 2:
18             resultado = e.n1 - e.n2;
19             operacion = "Resta";
20             cout << "Resta: " << resultado << endl;
21             break;
22
23         case 3:
24             resultado = e.n1 * e.n2;
25             operacion = "Multiplicacion";
26             cout << "Multiplicacion: " << resultado << endl;
27             break;
28
29         case 4:
30             if (e.n2 == 0)
31             {
32                 cout << "Error: no se puede dividir entre 0" << endl;
33                 return;
34             }
35
36             resultado = e.n1 / e.n2;
37             operacion = "Division";
38             cout << "Division: " << resultado << endl;
39             break;
40
41         default:
42             cout << "Opcion no valida" << endl;
43             return;
44     }

```

```

45
46 //CSV
47 ofstream csv("calculadora.csv", ios::app);
48
49 csv << e.n1 << ","
50     << e.n2 << ","
51     << operacion << ","
52     << resultado
53     << endl;
54
55 csv.close();
56
57 //TXT
58 ofstream txt("calculadora.txt", ios::app);
59
60 txt << "Numero1: " << e.n1 << endl;
61 txt << "Numero2: " << e.n2 << endl;
62 txt << "Operacion: " << operacion << endl;
63 txt << "Resultado: " << resultado << endl;
64 txt << "-----" << endl;
65
66 txt.close();
67
68 //XML
69 ofstream xml("calculadora.xml");
70
71 xml << "<operacion>" << endl;
72 xml << "    <numero1>" << e.n1 << "</numero1>" << endl;
73 xml << "    <numero2>" << e.n2 << "</numero2>" << endl;
74 xml << "    <tipo>" << operacion << "</tipo>" << endl;
75 xml << "    <resultado>" << resultado << "</resultado>" << endl;
76 xml << "</operacion>" << endl;
77
78 xml.close();
79
80 //JSON
81 ofstream json("calculadora.json");
82
83 json << "{" << endl;
84 json << "  \"numero1\": " << e.n1 << "," << endl;
85 json << "  \"numero2\": " << e.n2 << "," << endl;
86 json << "  \"operacion\": \"" << operacion << "\", " << endl;
87 json << "  \"resultado\": " << resultado << endl;
88 json << "}" << endl;

```

```

89
90 json.close();
91
92 //Confirmación de La creación
93 cout << endl;
94 cout << "Archivos generados correctamente:" << endl;
95 cout << "- calculadora.csv" << endl;
96 cout << "- calculadora.txt" << endl;
97 cout << "- calculadora.xml" << endl;
98 cout << "- calculadora.json" << endl;
99

```

Clase main.cpp

Coordina la ejecución del programa.

Secuencia:

1. Solicita los números
2. Muestra el menú
3. Ejecuta la operación seleccionada
4. Genera los archivos correspondientes

```
1 #include <iostream>
2 #include "IngresarNum.h"
3 #include "CalculadoraB.h"
4 #include "Operacion.h"
5
6 using namespace std;
7
8 int main()
9 {
10     IngresarNum e;
11     e.pedirNum(); // Pedimos n1 y n2
12
13     Operacion o;
14     o.mostrarMenu();
15
16     CalculadoraB p;
17     p.calcular(e, o.opcion); // Calculamos todas las operaciones
18
19     return 0;
20 }
```

Programa Lector

Clase Lector.h

Declara la clase encargada de abrir y mostrar el contenido de los archivos generados por la calculadora.

```
1 #ifndef LECTOR_H
2 #define LECTOR_H
3
4 class Lector
5 {
6 public:
7     Lector();
8 };
9
10 #endif
```

Clase Lector.cpp

Implementa el menú de selección de archivos.

Opciones:

1. CSV
2. TXT
3. XML
4. JSON

Después de seleccionar una opción:

- Abre el archivo correspondiente
- Lee línea por línea
- Muestra el contenido en pantalla

```
1 #include "Lector.h"
2 #include <iostream>
3 #include <fstream>
4 #include <string>
5
6 using namespace std;
7
8 Lector::Lector()
9 {
10     int opcion;
11     string nombreArchivo;
12
13     cout << "Seleccione el archivo a leer:" << endl;
14     cout << "1. CSV" << endl;
15     cout << "2. TXT" << endl;
16     cout << "3. XML" << endl;
17     cout << "4. JSON" << endl;
18
19     cin >> opcion;
20
21     // 1. Asignamos solo el nombre del archivo según la opción
22     switch(opcion)
23     {
24         case 1: nombreArchivo = "calculadora.csv"; break;
25         case 2: nombreArchivo = "calculadora.txt"; break;
26         case 3: nombreArchivo = "calculadora.xml"; break;
27         case 4: nombreArchivo = "calculadora.json"; break;
28         default:
29             cout << "Opcion no valida" << endl;
30             return;
31     }
32
33     // 2. Definimos las dos posibles rutas relativas (Escenarios)
34     string rutaEscenario1 = "../CalculadoraB/" + nombreArchivo;
35     string rutaEscenario2 = "../../Proyectos pasados/CalculadoraB/" + nombreArchivo;
36
37     // 3. Intentamos abrir primero el Escenario 1 (Carpetas sueltas juntas)
38     string rutaFinal = rutaEscenario1;
39     ifstream archivo(rutaFinal.c_str());
40
41     // 4. Si no se pudo abrir, intentamos de forma automática con el Escenario 2 (Estructura exclusiva de mi USB XD)
42     if(!archivo)
43     {
44         rutaFinal = rutaEscenario2;
```



```

45     archivo.open(rutaFinal.c_str());
46 }
47
48 // 5. Si ambos fallan, el archivo realmente no existe o el nombre de la carpeta está mal escrito
49 if(!archivo)
50 {
51     cout << "No se pudo abrir el archivo en ninguna de las ubicaciones relativas." << endl;
52     return;
53 }
54
55 // Lectura del archivo si se encontró con éxito
56 string linea;
57 cout << endl;
58 cout << "Contenido del archivo (" << rutaFinal << "):" << endl;
59 cout << "-----" << endl;
60
61 while(getline(archivo, linea))
62 {
63     cout << linea << endl;
64 }
65
66 archivo.close();
67 }

```

Clase main.cpp

Crea un objeto de la clase Lector e inicia la lectura del archivo seleccionado.

```

1  #include "Lector.h"
2
3  using namespace std;
4
5  int main()
6  {
7      Lector obj;
8
9
10
11     return 0;
12 }

```

Errores encontrados

Uso de rutas absolutas:

Inicialmente los archivos se almacenaban utilizando rutas específicas de la memoria USB.

Ejemplo:

"D:\Estructura de Datos\3\CSV\calculadora.csv"

Esto impedía que el programa funcionara correctamente en otras computadoras.

La solución fue utilizar rutas relativas:

“calculadora.csv”

Permitiendo que los archivos se generen en la misma carpeta del programa.

Error al utilizar string:

Al reemplazar varias variables por una variable de texto que almacena el nombre de la operación se presentó el error:

'string' does not name a type

La solución fue agregar la librería:

```
#include <string>
```

Y declarar:

```
std::string operacion;
```

Error en el historial CSV:

Al implementar el historial mediante:

```
ios::app
```

Los registros aparecían unidos debido a la ausencia de saltos de línea al final de cada operación.

La solución consistió en agregar:

```
<< endl;
```

Al finalizar cada registro almacenado.

Ejecución del programa

En el programa de calculadora, se ingresan los números 50 y 34 con la operación suma.

```
Ingrese el primer numero: 50
Ingrese el segundo numero: 34
Seleccione una operacion:
1. Suma
2. Resta
3. Multiplicacion
4. Division
1
Suma: 84

Archivos generados correctamente:
- calculadora.csv
- calculadora.txt
```

En el programa del lector, se revisa el archivo .csv.

```
Seleccione el archivo a leer:
1. CSV
2. TXT
3. XML
4. JSON
1

Contenido del archivo (../../Proyectos pasados/CalculadoraB/calculadora.csv):
-----
50,34,Suma,84
```

En la calculadora, se ingresan los números 136 y 45 con la operación resta.

```
Ingrese el primer numero: 136
Ingrese el segundo numero: 45
Seleccione una operacion:
1. Suma
2. Resta
3. Multiplicacion
4. Division
2
Resta: 91

Archivos generados correctamente:
- calculadora.csv
- calculadora.txt
- calculadora.xml
- calculadora.json
```

En el lector, se revisa el archivo .txt, para observar el historial, que debe funcionar igual que en el .csv (con respecto al guardar los datos).

```
Seleccione el archivo a leer:
1. CSV
2. TXT
3. XML
4. JSON
2

Contenido del archivo (../../Proyectos pasados/CalculadoraB/calculadora.txt):
-----
Numero1: 50
Numero2: 34
Operacion: Suma
Resultado: 84
-----
Numero1: 136
Numero2: 45
Operacion: Resta
Resultado: 91
-----
```

En la calculadora, se ingresan los números 95 y 43 con la operación de multiplicación.

```
Ingrese el primer numero: 95
Ingrese el segundo numero: 43
Seleccione una operacion:
1. Suma
2. Resta
3. Multiplicacion
4. Division
3
Multiplicacion: 4085

Archivos generados correctamente:
- calculadora.csv
- calculadora.txt
- calculadora.xml
- calculadora.json
```

En el lector se revisa el archivo .xml, que guarda únicamente el último elemento guardado.

```
Seleccione el archivo a leer:
1. CSV
2. TXT
3. XML
4. JSON
3

Contenido del archivo (../CalculadoraB/calculadora.xml):
-----
<operacion>
  <numero1>95</numero1>
  <numero2>43</numero2>
  <tipo>Multiplicacion</tipo>
  <resultado>4085</resultado>
</operacion>
```

Para comprobar que se guardaron los cambios, simplemente basta con revisar los archivos .csv o .txt.

```
Seleccione el archivo a leer:
1. CSV
2. TXT
3. XML
4. JSON
1

Contenido del archivo (../CalculadoraB/calculadora.csv):
-----
50,34,Suma,84
136,45,Resta,91
```

Como se puede observar en las rutas de acceso de los archivos, han cambiado con respecto a cada prueba, esto sirve para comprobar que el programa funciona con ambos parámetros.

Aclaración: Al estar ubicados en distintas direcciones, no comparten el mismo historial, para eso se deben realizar de nuevo las pruebas, o en su defecto, copiar todos los archivos a la respectiva ubicación.

Como última prueba, en la calculadora, se ingresan los números 59 y 12 con la operación de división.

```
Ingrese el primer numero: 59
Ingrese el segundo numero: 12
Seleccione una operacion:
1. Suma
2. Resta
3. Multiplicacion
4. Division
4
Division: 4.91667

Archivos generados correctamente:
- calculadora.csv
- calculadora.txt
- calculadora.xml
- calculadora.json
```

En el lector, se revisa el archivo .json, que como el .xml, solamente guarda el último registro.

```
Seleccione el archivo a leer:
1. CSV
2. TXT
3. XML
4. JSON
4

Contenido del archivo (../CalculadoraB/calculadora.json):
-----
{
  "numero1": 59,
  "numero2": 12,
  "operacion": "Division",
  "resultado": 4.91667
}
```

Para comprobar que se guardaron los cambios, simplemente basta con revisar los archivos .csv o .txt.

```
Seleccione el archivo a leer:
1. CSV
2. TXT
3. XML
4. JSON
2

Contenido del archivo (../CalculadoraB/calculadora.txt):
-----
Numero1: 50
Numero2: 34
Operacion: Suma
Resultado: 84
-----
Numero1: 136
Numero2: 45
Operacion: Resta
Resultado: 91
-----
Numero1: 95
Numero2: 43
Operacion: Multiplicacion
Resultado: 4085
-----
Numero1: 59
Numero2: 12
Operacion: Division
Resultado: 4.91667
-----
```

Con estas pruebas, se puede comprobar que los programas funcionan correctamente.

Programa del segundo parcial con Archivos:

1. Definición de la clase Alumno

```
// Clase Alumno
class Alumno {
public:
    int id;
    string nombre;

    Alumno();
    Alumno(int i, string n);
};
```

La clase **Alumno** representa la información de cada estudiante que será almacenado en el programa. Contiene dos atributos principales: un identificador (id) y un nombre (nombre). Además, cuenta con un constructor por defecto y un constructor parametrizado que permite crear objetos con valores específicos.

2. Definición de la clase Lista

```
// ADT Lista
class Lista {
private:
    Alumno* alumnos;
    int capacidad;
    int cantidad;

public:
    Lista(int cap = 10);
    ~Lista();

    void agregar(int id, string nombre);
    void mostrar();

    Alumno* obtenerArreglo();
    int getCantidad();
};
```

Descripción:

La clase **Lista** funciona como una Estructura de Datos Abstracta (ADT) encargada de almacenar los alumnos registrados. Utiliza un arreglo dinámico manejado mediante un puntero, permitiendo guardar varios objetos de la clase Alumno y controlar la cantidad de elementos almacenados.

3. Constructor de la lista

```
Lista::Lista(int cap) {  
    capacidad = cap;  
    cantidad = 0;  
    alumnos = new Alumno[capacidad];  
}  
  
Lista::~~Lista() {  
    delete[] alumnos;  
}
```

Este constructor inicializa la lista asignando una capacidad máxima, estableciendo que inicialmente no existen alumnos registrados y reservando memoria dinámica para el arreglo de objetos Alumno.

4. Método para agregar alumnos

```
void Lista::agregar(int id, string nombre) {  
    if (cantidad < capacidad) {  
        alumnos[cantidad] = Alumno(id, nombre);  
        cantidad++;  
    }  
}
```

Este método permite agregar un nuevo alumno al arreglo dinámico. Primero verifica que aún exista espacio disponible y posteriormente almacena el nuevo objeto en la siguiente posición libre, incrementando el contador de alumnos registrados.

5. Creación de archivos

```
// CREAR ARCHIVOS

void crearArchivos() {

    ofstream txt("datos.txt", ios::app);
    txt.close();

    ofstream csv("datos.csv", ios::app);
    csv.close();

    ofstream json("datos.json", ios::app);
    json.close();

    ofstream xml("datos.xml", ios::app);
    xml.close();
}
```

Esta función se ejecuta al iniciar el programa y tiene como objetivo verificar la existencia de los archivos TXT, CSV, JSON y XML. Si alguno no existe, se crea automáticamente para que posteriormente pueda utilizarse para almacenar información.

6. Guardado de información

```
void guardarTXT(Lista* lista) {

    ofstream archivo("datos.txt");

    Alumno* arr = lista->obtenerArreglo();

    for (int i = 0; i < lista->getCantidad(); i++) {

        archivo << arr[i].id
                << " "
                << arr[i].nombre
                << endl;
    }

    archivo.close();
}
```

Este fragmento escribe en el archivo todos los alumnos almacenados en la lista. Se recorre el arreglo dinámico y cada registro es enviado al archivo utilizando un flujo de salida (ofstream).

7. Lectura de archivos

```

void leerTXT() {

    ifstream archivo("datos.txt");

    if(!archivo)
    {
        cout << "No se pudo abrir datos.txt\n";
        return;
    }

    cout << "Archivo abierto correctamente\n";

    string linea;
    bool vacio = true;

    while(getline(archivo, linea))
    {
        cout << "[" << linea << "]" << endl;
        vacio = false;
    }

    if(vacio)
    {
        cout << "Archivo vacio\n";
    }

    archivo.close();
}

```

Esta función recupera la información almacenada previamente en el archivo y la muestra en pantalla. Para ello utiliza un flujo de entrada (ifstream) y lee cada línea mediante la función getline().

10. Tipos de archivos utilizados

TXT

Almacena la información en formato de texto plano, siendo el más sencillo de leer y editar.

CSV

Organiza los datos separados por comas, facilitando su uso en programas como Excel.

JSON

Guarda la información utilizando una estructura basada en objetos y pares clave-valor, muy utilizada en aplicaciones web y APIs.

XML

Representa la información mediante etiquetas jerárquicas, permitiendo organizar los datos de manera estructurada y legible.

Programa compilado:

```
in.cpp x SegundoParcial_Archivos.h x SegundoParcial_Archivos.cpp x
1 #include "SegundoParcial_Archivos.h"
2
3 int main() {
4     crearArchivos();
5
6     Lista* lista = new Lista();
7
8     int opcionArchivo;
9     int opcion;
10
11 }
```

```
C:\Users\franc\OneDrive\Doc x + v
Programa de archivos utilizando los temas del segundo parcial
Seleccione el tipo de archivo:
1. TXT
2. CSV
3. JSON
4. XML
```

El programa permite registrar y administrar información de alumnos utilizando una **Lista** como Estructura de Datos Abstracta (ADT). Cada alumno cuenta con un identificador y un nombre, los cuales son almacenados temporalmente en un arreglo dinámico manejado mediante punteros.

Al iniciar el programa, el usuario puede seleccionar el tipo de archivo con el que desea trabajar entre **TXT, CSV, JSON y XML**. Si los archivos no existen, el programa los crea automáticamente para garantizar su disponibilidad.

A través de un menú interactivo, el usuario puede agregar nuevos alumnos, visualizar la información almacenada en el archivo seleccionado y guardar los cambios realizados. El programa utiliza funciones de lectura y escritura para mantener la información de forma permanente, permitiendo recuperarla incluso después de cerrar y volver a ejecutar la aplicación.

Además, se implementan validaciones para verificar que los archivos se abran correctamente y para informar al usuario cuando un archivo no contiene información almacenada.

Ejercicio: Grafos

Este programa es un sistema en C++ para crear y manejar un Grafo (No Dirigido) por medio de una matriz de unos y ceros. El usuario puede dar de alta nodos (los lugares) y poner aristas (los caminos entre ellos) desde un menú.

Como es un grafo normal, el código hace que los caminos sean automáticamente de ida y vuelta, por lo que la matriz siempre queda simétrica (como un espejo). También incluye una opción para buscar rutas entre dos nodos, donde te muestra dos opciones: la más rápida (que es directa pero más cara) o la más barata (que tiene escalas pero es más larga). Por último, te deja guardar todo lo que hiciste en un archivo .txt y volverlo a cargar después para no perder tus datos, poniéndole la extensión .txt solito para que el usuario no tenga que escribirla.

Para iniciar tenemos la clase de Grafos

Aquí definimos las variables y arreglos privados para el control del grafo en la memoria, “n” es para contar los nodos para los nombres y la matriz para las aristas. Su constructor “Grafo()” se encarga de inicializar toda la matriz en ceros para limpiar los datos antes de empezar.

```
class Grafo
{
private:
    int n;
    string nodos[10];
    int matriz[10][10];

public:
    Grafo()
    {
        n = 0;
        for (int i = 0; i < 10; i++)
            for (int j = 0; j < 10; j++)
                matriz[i][j] = 0;
    }
}
```

Después pasamos al método de gestión de nodos.

```

void crearNodos()
{
    cout << "\n Cuantos nodos desea crear? ";
    cin >> n;
    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    for (int i = 0; i < n; i++)
    {
        cout << "Nombre del nodo " << i + 1 << ": ";
        getline(cin, nodos[i]);
    }
    cout << "\nNodos creados correctamente.\n";
}

int buscarNodo(string nombre)
{
    for (int i = 0; i < n; i++)
        if (nodos[i] == nombre)
            return i;
    return -1;
}

```

Aquí “crearNodos()” se encarga de solicitar al usuario la cantidad de nodos y capturar sus nombres.

Luego “buscarNodo()” recorre el arreglo de nombres de forma lineal para encontrar la posición exacta de un nodo y validar si existe o no en el sistema.

Continuamos y llegamos al método para la gestión de aristas simétricas.

Esta gestión se encarga de controlar las conexiones del grafo. Aquí “agregarArista()” activa el número 1 en ambas coordenadas cruzadas de la matriz. Y por otra parte “eliminarArista()” hace lo opuesto, asignando un 0 en ambos sentidos para romper la conexión.

```

void agregarArista()
{
    int veces;
    cout << "\n Cuantas aristas deseas agregar?: ";
    cin >> veces;
    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    for (int k = 0; k < veces; k++)
    {
        string origen, destino;
        cout << "\nArista " << k + 1 << endl;
        cout << "Nodo origen: ";
        getline(cin, origen);
        cout << "Nodo destino: ";
        getline(cin, destino);

        int i = buscarNodo(origen);
        int j = buscarNodo(destino);

        if (i == -1 || j == -1)
        {
            cout << "Uno de los nodos no existe.\n";
            continue;
        }

        matriz[i][j] = 1;
        matriz[j][i] = 1;
        cout << "Arista agregada en ambos sentidos.\n";
    }
}

```

```

void eliminarArista()
{
    string origen, destino;
    cout << "\nNodo origen: ";
    getline(cin, origen);
    cout << "Nodo destino: ";
    getline(cin, destino);

    int i = buscarNodo(origen);
    int j = buscarNodo(destino);

    if (i == -1 || j == -1)
    {
        cout << "Uno de los nodos no existe.\n";
        return;
    }

    matriz[i][j] = 0;
    matriz[j][i] = 0;
    cout << "\nArista eliminada correctamente en ambos sentidos.\n";
}

```

Después con el método de despliegue y consulta de rutas.

Este método imprime en la pantalla la matriz de adyacencia de forma ordenada, además, incluye el consultor interactivo que le pide al usuario un origen y un destino para simular y desplegar las dos opciones de rutas en pantalla, las cuales son:

La directa, rápida pero costosa o con escalas, es decir, económica pero larga.

```
void mostrar()
{
    if (n == 0)
    {
        cout << "\nNo hay nodos registrados.\n";
        return;
    }

    cout << "\n==== MATRIZ DE ADYACENCIA =====\n\n";
    cout << " ";
    for (int i = 0; i < n; i++)
        cout << nodos[i] << " ";
    cout << endl;

    for (int i = 0; i < n; i++)
    {
        cout << nodos[i] << " ";
        for (int j = 0; j < n; j++)
            cout << matriz[i][j] << " ";
        cout << endl;
    }

    // --- SECCIÓN: MOSTRAR LAS POSIBLES RUTAS ENCONTRADAS ---
    cout << "\n==== CONSULTOR DE RUTAS INTELIGENTE =====\n";
    string orig, dest;
    cout << "Ingrese Nodo de Origen para simular ruta: ";
    getline(cin, orig);
    cout << "Ingrese Nodo de Destino para simular ruta: ";
    getline(cin, dest);
}
```



```

int idx0 = buscarNodo(orig);
int idxD = buscarNodo(dest);

if (idx0 != -1 && idxD != -1 && matriz[idx0][idxD] == 1) {
    int submenu;
    cout << "\n Cual ruta deseas tomar?\n";
    cout << "1. Nodo " << orig << ", " << dest << " (La mas rapida, pero mas cara)\n";

    // Simulación dinámica de ruta larga intermedia según las letras
    if (orig == "A" && dest == "D") {
        cout << "2. Nodo A, B, C, D (La mas barata, pero mas larga)\n";
    } else {
        cout << "2. Nodo " << orig << ", B, " << dest << " (La mas barata, pero mas larga)\n";
    }

    cout << "Selecciona una opcion: ";
    cin >> submenu;
    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    if (submenu == 1) {
        cout << "\n-> RUTA SELECCIONADA: Directa y eficiente.\n";
    } else {
        cout << "\n-> RUTA SELECCIONADA: Ruta economica.\n";
    }
} else {
    cout << "\nNo existe una arista directa registrada entre esos nodos para evaluar rutas.\n";
}
}
}

```

Después utilizamos el método de persistencia de archivos. Es aquí en donde el flujo de datos hacia el disco duro “guardarArchivo()” exporta la configuración de los nodos y la matriz, mientras que “leerArchivo()” los importa para restaurar el grafo. Ambos métodos automatizan la extensión txt para evitar que el usuario deba escribirla de forma manual.

```

void guardarArchivo()
{
    string nombreArchivo;
    cout << "\nIngrese nombre del archivo (Se guardara como .txt de forma automatica): ";
    getline(cin, nombreArchivo);

    // Agregamos la extensión .txt automáticamente
    nombreArchivo = nombreArchivo + ".txt";

    ofstream archivo(nombreArchivo.c_str());
    if (!archivo)
    {
        cout << "Error al crear el archivo.\n";
        return;
    }

    archivo << n << endl;
    for (int i = 0; i < n; i++)
        archivo << nodos[i] << endl;

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
            archivo << matriz[i][j] << " ";
        archivo << endl;
    }
    archivo.close();
    cout << "Archivo '" << nombreArchivo << "' guardado correctamente.\n";
}

```

```

void leerArchivo()
{
    string nombreArchivo;
    cout << "\nIngrese nombre del archivo a cargar (Busqueda automatica de .txt): ";
    getline(cin, nombreArchivo);

    // Agregamos la extensión .txt automáticamente
    nombreArchivo = nombreArchivo + ".txt";

    ifstream archivo(nombreArchivo.c_str());
    if (!archivo)
    {
        cout << "El archivo '" << nombreArchivo << "' no existe.\n";
        return;
    }

    archivo >> n;
    archivo.ignore(numeric_limits<streamsize>::max(), '\n');

    for (int i = 0; i < n; i++)
        getline(archivo, nodos[i]);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            archivo >> matriz[i][j];

    archivo.close();
    cout << "\nArchivo '" << nombreArchivo << "' leído correctamente.\n";
}

```

Por último, tenemos la función principal, la función que controla el como arranca el programa. Contiene el menú interactivo de 7 opciones mediante un ciclo do-while y un switch-case e incorpora una protección con “cin.fail()” para así evitar que el sistema se trabe en un bucle si el usuario ingresa una letra en lugar de un número como se solicita.

```

int main()
{
    Grafo g;
    int opcion;

    do
    {
        cout << "\n=====";
        cout << "\n          SISTEMA DE GRAFOS";
        cout << "\n=====";
        cout << "\n1. Crear nodos";
        cout << "\n2. Agregar aristas";
        cout << "\n3. Eliminar arista";
        cout << "\n4. Mostrar grafo y consultar rutas";
        cout << "\n5. Guardar archivo (.txt automatico)";
        cout << "\n6. Leer archivo (.txt automatico)";
        cout << "\n7. Salir";
        cout << "\nOpcion: ";

        cin >> opcion;

        if (cin.fail())
        {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "\nOpcion invalida. Por favor introduzca un numero del 1 al 7.\n";
            opcion = 0;
            continue;
        }
    }
}

```

```

        cin.ignore(numeric_limits<streamsize>::max(), '\n');

        switch (opcion)
        {
            case 1: g.crearNodos(); break;
            case 2: g.agregarArista(); break;
            case 3: g.eliminarArista(); break;
            case 4: g.mostrar(); break;
            case 5: g.guardarArchivo(); break;
            case 6: g.leerArchivo(); break;
            case 7: cout << "\nPrograma finalizado.\n"; break;
            default: cout << "\nOpcion invalida.\n";
        }

    } while (opcion != 7);

    return 0;
}

```

Programa Dígrafos

El presente programa fue desarrollado en lenguaje C++ utilizando los principios de la Programación Orientada a Objetos (POO) y el concepto de Tipo Abstracto de Datos (ADT). Su propósito es permitir la creación y administración de un dígrafo, representando relaciones dirigidas entre diferentes nodos mediante rutas con un costo asociado.

Además de construir el dígrafo, el sistema permite determinar la mejor ruta disponible, guardar la información en diferentes formatos de archivo y consultar o modificar la información previamente almacenada.

El programa tiene como objetivo

- Crear nodos que representen los vértices del dígrafo.
- Crear rutas dirigidas entre los nodos.
- Asignar un costo a cada ruta.
- Mostrar gráficamente la estructura del dígrafo.
- Determinar la mejor ruta con base en el menor costo.
- Guardar la información en diferentes formatos de archivo.
- Leer y modificar archivos previamente creados.

```
#include <iostream>
#include <vector>
#include <string>
#include <limits>
#include <cstdio>
```

En esta parte se importan las bibliotecas necesarias para el funcionamiento del programa. La biblioteca `iostream` permite utilizar las instrucciones de entrada y salida de datos, como `cin` y `cout`, para que el usuario pueda interactuar con el sistema. La biblioteca `vector` se utiliza para almacenar una cantidad dinámica de elementos, en este caso los nodos y las rutas del dígrafo. La biblioteca `string` permite manejar cadenas de texto, como los nombres de los nodos y los nombres de los archivos. La biblioteca `limits` se utiliza para controlar y limpiar el búfer de entrada del teclado cuando es necesario. Finalmente, la biblioteca `cstdio` permite realizar operaciones con archivos mediante funciones del lenguaje C, como abrir, leer y guardar archivos. La instrucción `using namespace std;` evita tener que escribir el prefijo `std::` antes de cada elemento de la biblioteca estándar.

```
using namespace std;

// =====
// RUTA COMPLETA
// =====
struct Ruta
{
    vector<string> nodos;
    int costo;
};

class Digrafo
{
private:
    vector<string> nodosBase;
    vector<Ruta> rutas;

    string archivoActual;
    bool archivoCargado;

public:
    Digrafo()
    {
        archivoCargado = false;
        archivoActual = "";
    }
};
```

Se crea una estructura llamada `Ruta`, la cual representa un recorrido completo dentro del dígrafo. Esta estructura contiene un vector de cadenas denominado

nodos, donde se almacenan los nombres de los nodos que forman parte del recorrido, y una variable de tipo entero llamada costo, que almacena el costo total de esa ruta. Por ejemplo, una ruta puede estar formada por los nodos A, B y D y tener un costo de 10 unidades. Gracias a esta estructura, cada ruta puede almacenarse como un solo objeto, facilitando su administración dentro del programa.

```
// =====  
// CREAR NODOS  
// =====  
void crearNodos()  
{  
    int n;  
    cout << "\nCuantos nodos deseas crear: ";  
    cin >> n;  
    cin.ignore();  
  
    nodosBase.clear();  
  
    for (int i = 0; i < n; i++)  
    {  
        string nodo;  
        cout << "Nodo " << i + 1 << ": ";  
        getline(cin, nodo);  
        nodosBase.push_back(nodo);  
    }  
}
```

Este método tiene como finalidad crear los nodos que formarán parte del dígrafo. Primero se declara la variable entera n, la cual se utiliza para almacenar la cantidad de nodos que el usuario desea registrar. Mediante las instrucciones cout y cin, el programa solicita y recibe dicha cantidad. Posteriormente, la instrucción cin.ignore() limpia el búfer del teclado para evitar que el salto de línea generado por cin interfiera con la lectura de los nombres de los nodos.

La instrucción nodosBase.clear() elimina cualquier información que se encuentre previamente almacenada en el vector nodosBase. Esto permite que, si el usuario decide crear nuevos nodos, los anteriores sean reemplazados y se eviten datos duplicados o inconsistentes.

Después se utiliza una estructura repetitiva for, la cual se ejecuta tantas veces como nodos indicó el usuario. En cada iteración se declara una variable de tipo string llamada nodo, que almacena temporalmente el nombre del nodo que se está capturando. La instrucción getline(cin, nodo) permite leer el nombre completo del nodo, incluso si contiene espacios. Finalmente, la función push_back(nodo) inserta

el nombre capturado al final del vector `nodosBase`, almacenando cada nodo dentro de la estructura de datos del programa.

```
// =====  
// AGREGAR VARIAS RUTAS  
// =====  
void agregarRutas()  
{  
    int cant;  
    cout << "\nCuantas rutas deseas agregar: ";  
    cin >> cant;  
    cin.ignore();  
  
    for (int k = 0; k < cant; k++)  
    {  
        Ruta r;  
        int pasos;  
  
        cout << "\nRuta " << k + 1 << ":\n";  
  
        cout << "Cantidad de nodos en esta ruta: ";  
        cin >> pasos;  
        cin.ignore();  
  
        for (int i = 0; i < pasos; i++)  
        {  
            string nodo;  
            cout << "Nodo " << i + 1 << ": ";  
            getline(cin, nodo);  
            r.nodos.push_back(nodo);  
        }  
  
        cout << "Costo total de la ruta: ";  
        cin >> r.costo;  
        cin.ignore();  
  
        rutas.push_back(r);  
    }  
}
```

Este método tiene como finalidad registrar varias rutas del dígrafo en una sola operación. Primero se declara la variable entera `cant`, la cual almacena la cantidad de rutas que el usuario desea crear. Mediante `cout` y `cin`, el programa solicita este valor al usuario y posteriormente utiliza `cin.ignore()` para limpiar el búfer del teclado y evitar problemas en la captura de datos.

Después se utiliza un ciclo `for`, controlado por la variable `k`, el cual se ejecuta tantas veces como rutas indicó el usuario. En cada iteración se crea un objeto temporal de tipo `Ruta` llamado `r`, que servirá para almacenar la información de una ruta

específica. También se declara la variable entera pasos, que representa la cantidad de nodos que forman parte de dicha ruta.

El programa muestra el número de la ruta que se está registrando y solicita cuántos nodos la integran. Por ejemplo, si la ruta es:

A -> B -> C -> D

la cantidad de nodos o pasos sería igual a 4.

A continuación, se ejecuta un segundo ciclo for, controlado por la variable i, cuya función es capturar uno por uno los nodos que conforman la ruta. En cada repetición se declara una variable de tipo string llamada nodo, se solicita el nombre del nodo mediante getline() y se almacena dentro del vector nodos de la estructura Ruta utilizando la función push_back(). De esta manera, los nodos quedan guardados en el mismo orden en que forman el recorrido.

Una vez capturados todos los nodos, el programa solicita el costo total de la ruta y lo almacena en la variable costo perteneciente al objeto r. Finalmente, mediante la instrucción rutas.push_back(r), la ruta completa se agrega al vector rutas, quedando registrada dentro del sistema.

Por ejemplo, si el usuario desea agregar tres rutas:

Ruta	1:	A	->	B	->	D	Costo:	10		
Ruta	2:	A	->	B	->	C	->	D	Costo:	20
Ruta	3:	A	->	C	->	D	Costo:	15		

al finalizar la ejecución del método, el vector rutas almacenará tres objetos de tipo Ruta, cada uno con su secuencia de nodos y su respectivo costo. Esta información será utilizada posteriormente para mostrar el dígrafo, determinar la mejor ruta y guardar los datos en archivos.

```
// =====  
// MOSTRAR DIGRAFO COMO ÁRBOL  
// =====  
void mostrarDigrafo()  
{  
    cout << "\n==== DIGRAFO (ARBOL DE RUTAS) =====\n";  
  
    for (int i = 0; i < rutas.size(); i++)  
    {  
        cout << "\nRuta " << i + 1 << " | costo " << rutas[i].costo << "\n";  
  
        for (int j = 0; j < rutas[i].nodos.size(); j++)  
        {  
            cout << rutas[i].nodos[j];  
  
            if (j != rutas[i].nodos.size() - 1)  
                cout << " -> ";  
        }  
  
        cout << endl;  
    }  
}
```

El método tiene como objetivo mostrar el dígrafo como un “árbol de rutas”, es decir, presenta de forma ordenada todos los caminos almacenados junto con su costo. Primero imprime un título para indicar que se mostrará esta información y luego recorre todas las rutas guardadas en un vector, mostrando el número de cada ruta y su costo asociado.

Dentro de cada ruta, se recorre otro ciclo para imprimir los nodos que la componen en orden. Entre cada nodo se coloca una flecha “->” para representar la dirección del recorrido, evitando ponerla después del último nodo. Al final, el usuario puede visualizar claramente cada camino posible del dígrafo y entender cómo se conectan los nodos y cuánto cuesta recorrer cada ruta.

```
// =====  
// MEJOR RUTA  
// =====  
void mejorRuta()  
{  
    if (rutas.empty())  
    {  
        cout << "\nNo hay rutas.\n";  
        return;  
    }  
  
    int best = 0;  
  
    for (int i = 1; i < rutas.size(); i++)  
        if (rutas[i].costo < rutas[best].costo)  
            best = i;  
  
    cout << "\n==== MEJOR RUTA ==== \n";  
    cout << "Costo mínimo: " << rutas[best].costo << "\n";  
    cout << "Camino: ";  
  
    for (int i = 0; i < rutas[best].nodos.size(); i++)  
    {  
        cout << rutas[best].nodos[i];  
        if (i != rutas[best].nodos.size() - 1)  
            cout << " -> ";  
    }  
  
    cout << endl;  
}
```

Este método tiene como función principal analizar todas las rutas almacenadas en el dígrafo y determinar cuál es la mejor, entendida como la que tiene el menor costo total. Es decir, compara los distintos caminos registrados y selecciona el más eficiente en términos de peso o costo.

Primero verifica si no hay rutas guardadas; si el vector está vacío, muestra un mensaje y termina la ejecución para evitar errores. Luego asume inicialmente que la primera ruta es la mejor y recorre todas las demás comparando sus costos. Cada vez que encuentra una ruta con menor costo, actualiza la “mejor” registrada. Al final, muestra en pantalla el costo mínimo y el camino completo de esa ruta, imprimiendo los nodos en orden con flechas entre ellos para representar la dirección del

recorrido. Así, el método no solo compara, sino que también traduce los datos en una ruta óptima clara y fácil de interpretar.

```
// =====  
// GUARDAR CON FORMATO  
// =====  
void guardar()  
{  
    string nombre;  
    int op;  
  
    cout << "\nNombre del archivo: ";  
    cin >> nombre;  
  
    cout << "\nFormato:\n1 TXT\n2 CSV\n3 JSON\n4 HTML\n5 CC\nOpcion: ";  
    cin >> op;  
  
    string file = nombre;  
  
    if (op == 1) file += ".txt";  
    if (op == 2) file += ".csv";  
    if (op == 3) file += ".json";  
    if (op == 4) file += ".html";  
    if (op == 5) file += ".cc";  
  
    FILE* f = fopen(file.c_str(), "w");  
  
    for (int i = 0; i < rutas.size(); i++)  
    {  
        fprintf(f, "Ruta %d | costo %d | ", i + 1, rutas[i].costo);  
  
        for (int j = 0; j < rutas[i].nodos.size(); j++)  
        {  
            fprintf(f, "%s", rutas[i].nodos[j].c_str());  
            if (j != rutas[i].nodos.size() - 1)  
                fprintf(f, " -> ");  
        }  
    }  
}
```

Este método tiene como función guardar la información del dígrafo en un archivo, permitiendo exportar las rutas y sus costos en distintos formatos para su posterior consulta o modificación. En otras palabras, transforma los datos del programa en un documento externo que puede conservarse. Primero solicita al usuario el nombre del archivo y la opción de formato (TXT, CSV, JSON, HTML o CC). Con base en la opción elegida, se le agrega la extensión correspondiente al nombre del archivo. Luego se crea o abre el archivo en modo escritura. Después, el método recorre todas las rutas almacenadas: por cada una escribe su número y su costo, y dentro de otra iteración escribe los nodos que la forman, colocando flechas entre ellos para representar el recorrido. Al finalizar, cierra el archivo para asegurar que la información se guarde correctamente y muestra un mensaje confirmando el nombre

del

archivo

creado

```
// LEER ARCHIVO (CON EXTENSIÓN)
// =====
void leerArchivo()
{
    cout << "\nNombre del archivo con extension: ";
    cin >> archivoActual;

    FILE* f = fopen(archivoActual.c_str(), "r");

    if (!f)
    {
        cout << "No se pudo abrir.\n";
        archivoCargado = false;
        return;
    }

    archivoCargado = true;

    cout << "\n===== CONTENIDO DEL ARCHIVO =====\n";

    char linea[300];
    while (fgets(linea, sizeof(linea), f))
        cout << linea;

    fclose(f);

    char op;
    cout << "\n¿Deseas modificar el archivo? (s/n): ";
    cin >> op;

    if (op == 's' || op == 'S')
    {
        modificarArchivo();
    }
}
```

Este método tiene como función abrir un archivo previamente guardado y mostrar su contenido en pantalla, permitiendo al usuario consultar las rutas y, si lo desea, modificarlas después. Para ello, solicita el nombre del archivo con su extensión y lo guarda como el archivo actualmente activo.

Luego intenta abrirlo en modo lectura; si no existe o no puede abrirse, muestra un error y detiene la ejecución. Si la apertura es exitosa, marca que hay un archivo cargado y procede a mostrar un encabezado antes de leer su contenido. El archivo se recorre línea por línea usando lectura de texto, imprimiendo cada línea exactamente como está almacenada. Después de mostrar todo, el archivo se cierra para liberar recursos.

Finalmente, el método pregunta al usuario si desea modificar la información; si responde afirmativamente, se activa el proceso de modificación. Así, este procedimiento no solo permite visualizar los datos del dígrafo guardados, sino

también abrir la puerta a su edición controlada, asegurando primero que el archivo exista y haya sido leído correctamente.

```
// =====  
// MODIFICAR + RE-GUARDAR  
// =====  
void modificarArchivo()  
{  
    if (!archivoCargado)  
    {  
        cout << "\nPrimero lee un archivo.\n";  
        return;  
    }  
  
    cout << "\n1 Agregar ruta\n2 Eliminar ultima ruta\n0pcion: ";  
    int op;  
    cin >> op;  
  
    if (op == 1)  
    {  
        agregarRutas();  
    }  
    else if (op == 2 && !rutas.empty())  
    {  
        rutas.pop_back();  
    }  
  
    cout << "\nArchivo modificado en memoria.\n";  
  
    char save;  
    cout << "¿Deseas guardar cambios? (s/n): ";  
    cin >> save;  
  
    if (save == 's' || save == 'S')  
    {  
        FILE* f = fopen(archivoActual.c_str(), "w");
```

Este método permite modificar el contenido del dígrafo que ya fue cargado previamente y, si el usuario lo decide, guardar esos cambios en el archivo original. Su función es actualizar rutas sin necesidad de crear un archivo nuevo desde cero.

Primero verifica si hay un archivo cargado; si no existe, detiene el proceso para evitar errores. Luego muestra un menú donde el usuario puede agregar una nueva ruta o eliminar la última existente. Si elige agregar, se llama a la función correspondiente; si elige eliminar, se borra el último elemento del vector de rutas, siempre y cuando no esté vacío.

Después de aplicar cambios en memoria, el sistema pregunta si desea guardarlos. Si la respuesta es afirmativa, abre el archivo en modo escritura y vuelve a recorrer todas las rutas almacenadas, escribiendo cada una con su costo y sus nodos en formato ordenado con flechas entre ellos. Finalmente cierra el archivo y confirma

que los cambios fueron guardados correctamente, asegurando que la información del dígrafo quede actualizada de forma permanente.

En conclusión, este método permite controlar de manera segura el proceso de modificación de los archivos del sistema, ya que primero verifica que exista un archivo abierto, después permite agregar o eliminar rutas del dígrafo y, finalmente, ofrece la posibilidad de guardar permanentemente los cambios realizados

Este sistema construye un dígrafo que no se queda quieto: respira, se guarda, se abre y se transforma. Permite registrar rutas con sus nodos y costos, mostrarlas de forma clara, comparar caminos para encontrar el más eficiente y, si hace falta, exportar toda la información a distintos formatos de archivo. También puede volver a abrir esos datos, revisarlos y modificarlos sin perder el control de lo ya almacenado.

En conjunto, el programa no solo almacena información: la organiza, la interpreta y la conserva. Es como un mapa vivo donde cada ruta cuenta una historia distinta, y el sistema decide cuál es la más conveniente. Al final, su valor está en algo simple pero poderoso: dar orden a lo complejo y permitir que los datos del dígrafo puedan consultarse, analizarse y actualizarse sin romper su estructura.